

Prompt Library — Coding Task Examples

Sixteen example prompts covering the task types dev teams hit most often, written as plain `.txt` files — the way you'd naturally type them straight into Claude Chat: plain sentences plus simple numbered lists, no headers, no tables, no special formatting.

Tier suggestions below are a starting point based on scope and reasoning depth, not a hard rule — your router's rule/semantic layers will adjust based on actual length, doc count, and code volume.

File	Task type	Suggested tier	Why
<code>01-research.txt</code>	Research / comparison	Sonnet	Moderate synthesis across a few known options
<code>02-excel-requirements-doc.txt</code>	Requirements doc generation	Sonnet–Opus	Scales with number of functional areas / documents involved
<code>03-drawio-diagram.txt</code>	Diagram generation	Sonnet	Structured output, moderate reasoning about layout
<code>04-edit-few-lines.txt</code>	Small, scoped fix	Haiku	Tiny, unambiguous, single function
<code>05-create-entire-file.txt</code>	New file, following existing patterns	Sonnet	Moderate — needs to match conventions but is single-file
<code>06-create-entire-project.txt</code>	Full project scaffold	Opus	Many interacting files, architectural decisions
<code>07-edit-entire-file.txt</code>	Full-file rewrite	Opus	Behavior-preserving rewrite across an entire file is high-risk
<code>08-edit-multiple-files.txt</code>	Cross-cutting multi-file change	Opus	Coordination across files + a new migration + risk analysis
<code>09-debugging-bug-fix.txt</code>	Debugging	Sonnet	Root-cause reasoning from a stack trace, single function fix
<code>10-code-review.txt</code>	Code review	Sonnet	Judgment-heavy but bounded in scope
<code>11-write-tests.txt</code>	Test generation	Sonnet	Needs to reason about edge cases, not just boilerplate
<code>12-documentation.txt</code>	Docs / docstrings	Haiku–Sonnet	Mechanical for docstrings, light reasoning for the README section
<code>13-glue-code-integration.txt</code>	Integration / webhook code	Sonnet	Security-sensitive but scoped to one endpoint
<code>14-security-review.txt</code>	Security audit	Opus	High-stakes, needs deep, careful reasoning
<code>15-performance-optimization.txt</code>	Performance fix	Sonnet–Opus	Depends on how subtle the bottleneck is
<code>16-database-schema-migration.txt</code>	Schema/migration design	Opus	High-risk, needs to reason about failure modes and rollout

Why .txt and not .md

None of these need markdown-specific rendering (headers, links, tables) — just plain paragraphs and numbered lists, which look and work identically whether the file is `.txt` or `.md`. So `.txt` is the honest, simplest choice here. The model router (and Claude Chat itself) handles either one the same way, so use `.md` instead if you genuinely want to preserve things like links or tables in a document.

Using these with the model router

The short one-line versions of these task types are exactly what `router/exemplars.yaml` expects. If you want the router to recognize these categories specifically, add condensed one-sentence versions under the matching tier in that file — the fuller files here are better used as prompt *templates* you fill in and paste as your actual `prompt.txt` for a given task.